

MoCHII User Guide

Abstract

How to build and run MoCHII (MOnte Carlo for H II regions): requirements, the complete input-parameter reference, output-file formats, worked examples keyed to the regression tests, and the runbook for adding an ion. The physics and validation are documented separately in docs/MoCHII_physics.pdf; the atomic-data fitting pipeline in docs/MoCHII_fitting.pdf.

Contents

1. Requirements and build	1
2. Input parameters	2
2.1 Photons and RNG	2
2.2 Grid and dust density	4
2.3 Ionizing/FUV band and the source	5
2.4 Initial state and iteration	6
2.5 Convergence of the gas iteration	6
2.6 Thermal balance and metals	7
2.7 I-front re-refinement	8
2.8 Dust in the band and dust emission	9
2.9 Peel-off imaging	10
2.10 Output	10
3. Output files	10
3.1 <base>_rates.h5	10
3.2 Text outputs	11
3.3 Reading outputs in Python	11
3.4 Slices and cutouts	12
3.5 Building AMR grids	13
4. Worked examples (the regression tests)	13
5. Adding an ion	14
6. Practical notes	15

1. Requirements and build

- Intel oneAPI MPI Fortran (mpiifort/mpiifx; GNU mpif90 supported by the Makefile), HDF5 (default prefix /data/opt/hdf5_intel; override with make HDF5_PREFIX=...), CFITSIO.

- The SEDust dust-emission library, self-contained under SEDust/ in this tree. Build it once with `cd SEDust/sed && ./build_lib.sh` (→ SEDust/sed/lib/libsedust.a and its .mod files, which the Makefile links). The SEDust *data* directory is read at run time and resolved automatically relative to the executable (Section 2.8).
- Python 3 with numpy/scipy (h5py, astropy, matplotlib, PyNeb for the test and pipeline scripts).

Build:

```
cd MoCHII
(cd SEDust/sed && ./build_lib.sh) # on a fresh checkout
make # -> MoCHII.x (always a full clean rebuild)
make DEBUG=1 # bounds/traceback build
```

The SEDust library only needs the one-time `build_lib.sh` step; afterward `make` links the in-tree SEDust library. The Makefile intentionally performs a full rebuild every time (the MoCafe policy): there is no inter-module dependency tracking, and stale .mod files after editing `define.f90` otherwise cause runtime namelist errors.

Run:

```
mpirun -np 8 ./MoCHII.x input.in
```

MoCHII is MPI-only; the octree and all leaf arrays live in MPI-3 shared memory (one copy on each node).

2. Input parameters

All input is one `¶meters` namelist assigning fields of `par`. Parameters not listed keep the defaults shown. The switches compose: `gas_niter = 0` gives a single rates-only pass; `gas_niter > 0` iterates the ionization; `solve_te` adds the thermal balance; `use_metals` the trace-metal registry; `diffuse_field` the explicit diffuse field (forces case A); `refine_front` the I-front re-refinement; `ion_add_dust` the dust coupling.

2.1 Photons and RNG

parameter	default	meaning
no_photons	1e5	source packets in each iteration
no_print	1e7	progress-print stride
iseed	0	RNG seed (0 = from clock/pid)

2.2 Grid and dust density

parameter	default	meaning
grid_type	'car'	'amr' (octree) or 'car' (single-level Cartesian, leaf list in raster order; no tree and no geometry arrays in memory — cell centers and half-widths are computed analytically from the raster index; refine_front unavailable). 'uniform' is accepted as an alias for 'car'
car_walk	'dda'	car-grid traversal: 'dda' (default) = dedicated incremental Amanatides–Woo walks ($t_{\max}/t_{\Delta}$ per axis, comparisons and additions only — the fastest car walk); 'shared' = the octree walks with integer index arithmetic replacing the neighbor table, a verification mode that is bit-identical to 'amr'
amr_file	''	generic AMR leaf file (FITS/HDF5/text): columns x, y, z, level, nH [, metallicity, xHI, ndust]. If empty, a 'car' grid is built from the namelist instead ('amr' requires the file)
nx, ny, nz	1, 1, 11	'car'-from-namelist cell counts (cubic cells required: $2x_{\max}/n_x = 2y_{\max}/n_y = 2z_{\max}/n_z$)
xmax, ymax, zmax	1, 1, 1	'car'-from-namelist box half-extents [distance_unit]; box is $[-x_{\max}, x_{\max}]$ etc.
nH_const	-1	if ≥ 0 , override every leaf's n_{H} with this uniform value [cm^{-3}] (both 'car' and 'amr'); < 0 keeps the file/column density
density_file	''	read the per-leaf n_{H} from a uniform 3D density cube onto a 'car' grid (FITS 3D image, or the MoCHII HDF5 image format; NAXIS1/2/3 = $n_x/n_y/n_z$, values n_{H} [cm^{-3}]). $n_x/n_y/n_z$ come from the file, the box is xmax/ymax/zmax; an alternative to amr_file. reduce_factor > 1 down-samples; a rmax/rmin cut can mask the cube to a sphere
rmax, rmin	-999, 0	spherical density cut on leaf centers (both grids): $n_{\text{H}} = 0$ for $r > r_{\max}$ ($r_{\max} > 0$) and for $r < r_{\min}$ ($r_{\min} > 0$; a shell); radii in distance_unit
distance_unit	'kpc'	'kpc'/'pc'/'au'/'' (code units)
dust_model	'global_dgr'	leaf grey opacity: 'none', 'global_dgr' ($n_{\text{H} \text{cext}}$, DGR), 'from_file' (ndust column), 'laursen09' (static xHI column), 'laursen09_live' (computed x_{HI} , refreshed each iteration)
cext_dust	1.6059e-21	reference extinction per H [cm^2] at lambda_ref
DGR, Z_global, Z_ref	1e-2, 0.0134, 0.0134	dust-model scalings
f_ion_dust	0.01	laursen09 survival of dust in ionized gas
f_pah, f_ion_pah	0, 0	PAH share of the reference extinction and its own ionized-gas survival (laursen09_live)

2.3 Ionizing/FUV band and the source

parameter	default	meaning
use_ion_band	F	must be T (the MoCHII transport)
nnu_ion	16	ionizing-band bins; gates use 32. With ion_align_edges on (the default) the bins are log-uniform within each threshold-bounded segment; off, they are a single log grid
eion_min, eion_max	13.598, 100	ionizing band edges [eV]
ion_align_edges	T	pin the ionizing-band bin edges onto the ionization thresholds (H I 13.6, He I 24.6, He II 54.4 eV plus the active metal edges) so no bin straddles one; segments between thresholds are log-uniform, and if the thresholds outnumber the bins the lowest-priority metal edges are dropped (H/He always kept), so it is safe at any nnu_ion. Removes the $\sim 18\%$ He-ionizing-photon excess of a coarse log grid crossing the 24.6 eV edge. Set F for the legacy single log grid
add_fuv	F	FUV extension: extra log bins on [efuv_min, eion_min] below the unchanged ionizing bins; transports FUV grain heating and FUV photoionization of low-threshold metals (Mg I, C I, Si I, Fe I); luminosity stays the IONIZING luminosity (Q_H preserved, the FUV part rides on top)
efuv_min	6.0	FUV lower edge [eV]
nnu_fuv	8	FUV bins (16 recommended with metals)
tstar	-999	Planck source temperature [K]
ion_spectrum	"	alternative 2-column spectrum file (E [eV], L_E arb.)
luminosity	1.0	BAND luminosity [erg/s]
xs_point, ys_point, zs_point	0	point-source position (code units, recentered box)

2.4 Initial state and iteration

parameter	default	meaning
xHI_init	1.0	initial $n_{\text{HI}}/n_{\text{H}}$ (an xHI file column wins); an ionized start (1e-3) converges much faster than neutral
xHeI_init, xHeII_init	1, 0	initial He fractions
He_abund	0.1	$n_{\text{He}}/n_{\text{H}}$
gas_niter	0	iterations (0 = a single rates-only pass)
gas_tol	1e-3	convergence tolerance on x_{HII} (measure set by conv_crit)
conv_crit	'cell'	convergence measure: 'cell' (cell maxima; stalls at the front-cell noise floor) or 'vol' (volume-integrated L_1 ; noise-robust, recommended). See §2.5
ion_relax	1.0	under-relaxation on x (< 1 damps front oscillation)
case_ab	'B'	case A/B recombination ('A' forced by the diffuse field); also selects the SH95 H I/He II line tables (case-A/B)
require_convergence	F	if the gas iteration has not converged at gas_niter, print a warning; when T, abort before writing output. The convergence status is recorded in the _rates header either way (CONVERGD, NITERDON, FINALDX, FINALDTE)

2.5 Convergence of the gas iteration

The gas iteration (gas_niter sweeps of transport $\rightarrow$ rate integrals $\rightarrow$ per-leaf solve $\rightarrow$ opacity refill) stops when the state stops changing between sweeps. Each sweep compares a *change measure* against a tolerance:

- ionization only (te_fixed, no thermal solve): converged when the x_{HII} change $<$ gas_tol;
- coupled (solve_te): converged when the x_{HII} change $<$ gas_tol *and* the T_e change $<$ gas_tol_te.

Two measures (conv_crit). Both are computed every sweep; conv_crit selects which one gates the loop.

'cell' the cell maxima $\max_{\text{leaf}} |\Delta x_{\text{HII}}|$ and $\max_{\text{leaf}} |\Delta T_e|/T_e$. Simple, but the ionization-front cells jitter with Monte Carlo noise and never settle, so $\max |\Delta x_{\text{HII}}|$ *stalls* at the front-cell noise floor ($\sim 2\text{--}4 \times 10^{-2}$ at 4×10^7 packets) and a smaller gas_tol is unreachable. This is the historical default (kept so the recorded gates reproduce).

'vol' the volume-integrated L_1 changes

$$\frac{\sum_{\text{leaf}} V |\Delta x_{\text{HII}}|}{\sum_{\text{leaf}} V x_{\text{HII}}} < \text{gas_tol}, \quad \frac{\sum_{\text{leaf}} V n_e |\Delta T_e|}{\sum_{\text{leaf}} V n_e T_e} < \text{gas_tol_te}.$$

Front jitter occupies little volume (a few thousand front cells against $\sim 10^5$ ionized ones), so it enters the norm suppressed by that ratio; the n_e weight drops the vacuum and te_min-pinned no-heating cells that otherwise dominate $\max |\Delta T_e|$ in FUV/PDR runs. These measures decay smoothly — the recommended production setting.

The Monte Carlo floor. Even the volume measures do not reach zero: they floor at a Monte Carlo level set by the packet count, scaling as $1/\sqrt{N_{\text{packets}}}$ ($\sim 3 \times 10^{-3}$ in x and $\sim 7 \times 10^{-3}$ in T_e at 8×10^6 packets). Reaching the floor is

convergence — iterating further leaves the solution unchanged. So **set `gas_tol/gas_tol_te` just above the floor for the packet count in use**: a tolerance below the floor can never be met and the loop runs to `gas_niter` and warns. Rules of thumb (`conv_crit='vol'`): $\sim 5 \times 10^{-3}/10^{-2}$ at $\sim 10^7$ packets, loosened to $\sim 10^{-2}/2 \times 10^{-2}$ at $\sim 10^6$ (the reduced-count examples/ use the looser values for this reason).

When it does not converge. Hitting `gas_niter` without meeting the test is not fatal: the state is still written and a warning is printed on rank 0. Set `require_convergence = T` to abort before the output instead. Either way the `_rates` header records the outcome in `CONVERGD` (0/1), `NITERDON` (sweeps run), `FINALDX`, and `FINALDTE`. The derivation and the gate measurements are in `docs/MoCHII_physics.pdf`.

2.6 Thermal balance and metals

parameter	default	meaning
<code>solve_te</code>	F	bisection on T_e coupled with the ionization
<code>te_fixed</code>	1e4	fixed T_e [K] when <code>solve_te</code> is off
<code>te_min, te_max</code>	1e3, 5e4	bisection bracket [K]
<code>gas_tol_te</code>	1e-3	convergence on $\max \Delta T_e /T_e$
<code>atomic_dir</code>	'data/atomic'	coefficient files (relative to the run directory)
<code>use_metals</code>	F	registry cascade + metal line cooling
<code>ci_model</code>	'voronov'	collisional ionization: 'voronov' (default) or 'dere_hybrid' (Voronov $\times$ the Dere 2007/Voronov ratio; matters only in shielded low-ionization zones)
<code>abund_C/N/O/Ne/S</code>	HII20 values	$n(X)/n(H)$; an element is active when > 0
<code>abund_Ar/Mg/Fe</code>	0	further registry elements (gas-phase values; Mg/Fe are heavily depleted)
<code>abund_Si/Cl/Ca</code>	0	five-stage registry elements (gas-phase; Si and especially Ca heavily depleted, Cl nearly undepleted — Orion-like $3e-6 / 3e-7 / 2e-8$); adds [Si II] 34.8 μm , Si III/IV, [Cl II/III/IV] (5518/5538 density pair), Ca II, [Ca V]
<code>ion_metal_abs</code>	T	metal photoionization absorption in the band opacity (active with <code>use_metals</code>); the only gas opacity in the FUV bins

parameter	default	meaning
emis_output	F	write <code>_emis</code> (HDF5/FITS): leaf-by-leaf line emissivities + the state needed for maps (Section 3.)
h_coll_effects	F	add the collisional H I Balmer component (excitation of neutral H) as separate hc rows/blocks; matters at fronts and hot cells
fe_levels_full	F	load the expanded Fe II/III models (<code>nlevel_fe_{2,3}_full.txt</code> , 16/34 levels) instead of the compact 13/14 for iron-line studies
diffuse_field	F	explicit ground-recombination packets
hei_diffuse	F	fourth diffuse channel (needs <code>diffuse_field</code>): He I excited-level recombination photons (19.82 eV, 584 Å, two-photon) — correct energy bookkeeping; moves Te by ~1% at HII40, He split unchanged
gaunt_vh14	F	free-free Gaunt factors from the van Hoof et al. (2014) table (<code>data/gauntff_vh14.dat</code>) instead of Hummer (1988); the two agree to $\lesssim 0.5\%$ in T_e
metal_ne	F	PDR: electrons from the metal cascade join the n_e closure (the only electrons beyond the I-front)
metal_heat	F	PDR: metal photoheating in the thermal balance
grain_pe	F	PDR thermal package: grain photoelectric heating + recombination cooling (Bakes & Tielens 1994) from the local FUV field (needs <code>add_fuv</code>), plus H-impact [C II] 158 μm / [O I] 63 μm cooling; scaled by <code>pe_scale</code> and the leaf dust amount
pe_scale	1.0	photoelectric-efficiency scale factor

2.7 I-front re-refinement

parameter	default	meaning
refine_front	F	rebuild the octree on the I-front once
refine_iter	20	at which iteration
refine_lbase, refine_lmax	5, 7	base/front levels
refine_eps	1e-2	front flag: $\epsilon < x_{\text{HI}} < 1-\epsilon$
refine_dx	0.2	or a face-neighbor x_{HI} jump above this

2.8 Dust in the band and dust emission

parameter	default	meaning
ion_add_dust	F	dust absorption in the band (dusty Strömgren)
ion_dust_kext	"	kext table with EUV coverage (e.g. data/kext_albedo_WD_MW_3.1_60_D03.all_2009); required
ion_dust_scatter	F	sample dust scattering (HG, albedo weights)
dust_sed	F	SEDust stochastic dust SED at output time
dust_model_sed	'astrodust'	'astrodust'/'dl07'/'zubko'
sed_workdir	" (auto)	SEDust sed/ tree whose dielectric tables are read relative to it; blank auto-resolves to <executable dir>/SEDust/sed (the in-tree copy), so it normally needs no setting
sed_qtable, sed_sizedist	in-tree defaults	optics/size-distribution tables (relative to sed_workdir)
sed_NT, sed_Tlo, sed_Thi	200, 2.7, 5e3	SEDust temperature grid
dust_emis_bands	unset	up to 8 wavelengths [μm] (e.g. 8, 24, 70, 160): SEDust band emissivities of each leaf $\rightarrow$ _dustemis (blocks emis_dust/wl_dust [$\text{erg s}^{-1} \text{cm}^{-3} \mu\text{m}^{-1}$]); the Python map maker projects them into dust-band images (the IR is optically thin)
sed_pah_live	F	x_{HII} -dependent PAH survival in the emission: the leaf spectrum is assembled from the model channels with the PAH channel weighted by $(x_{\text{HI}} + f_{\text{ion,pah}} x_{\text{HII}})$ (needs a model with a 'PAH' channel, i.e. astrodust); turns the 8- μm map into the classic PAH ring at the ionization front

2.9 Peel-off imaging

parameter	default	meaning
ion_peel	F	after convergence, one extra transport pass peels direct (stellar + diffuse) and dust-scattered contributions toward the observers $\rightarrow$ <code>_image</code> (blocks <code>direct_ion/scatt_ion</code> , plus <code>_fuv</code> with <code>add_fuv</code>) [$\text{erg s}^{-1} \text{cm}^{-2} \text{sr}^{-1}$]
peel_bins	F	bin-resolved cubes <code>direct_cube/scatt_cube</code> (n_x, n_y, n_ν) in addition to the band-integrated channel images (hardness maps, synthetic EUV/FUV photometry); the third (bin) axis is labelled by the <code>E_bin/dE_bin</code> [eV] and <code>wl_bin</code> [$\text{\AA}$] arrays written once into the same file
save_direct0	F	also write the unattenuated direct image (<code>direct0_ion/_fuv</code> , and <code>direct0_cube</code> with <code>peel_bins</code>): the direct source with the path extinction $e^{-\tau}$ omitted, so <code>direct0/direct</code> = $e^{+\tau}$ is the line-of-sight optical depth toward the observer
obsx, obsy, obsz	unset	observer positions (or directions) in code units; or use <code>alpha/beta/gamma</code> angle lists [deg]; default: one observer on +z far away
distance	auto	observer distance (code units)
nxim, nyim	129, 129	image pixels
dxim, dyim	auto	pixel scale [deg]; auto-sized to cover the box

2.10 Output

`out_file` (base name; extension follows `file_format` = 'hdf5' or 'fits').

3. Output files

3.1 <base>_rates.h5

One dataset for each extension, leaf-ordered:

extension	content
Gamma_HI/HeI/HeII	photoionization rates [s^{-1}]
Heat_HI/HeI/HeII	photoheating per particle [erg/s]
Heat_dust	grain heating [$\text{erg s}^{-1} \text{cm}^{-3}$] (when ion_add_dust)
T_dust	equilibrium mixture dust temperature [K]
x_HI, x_HeI, x_HeII, n_e, T_e	converged state
x_<el>_stages	metal ion fractions (stage, leaf)
J_nu	band mean intensity (bin, leaf) [$\text{erg/s/cm}^2/\text{Hz/sr}$]
E_bin, dE_bin	band grid [eV]
LeafXYZ, LeafSize	leaf centers and full widths (code units; for slices/cutouts)

The file header also carries the convergence record of the gas iteration: CONVERGD (did it converge), NITERDON (iterations run), FINALDX (final max $|\Delta x_{\text{HII}}|$), and FINALDTE (final max $|\Delta T_e|/T_e$).

3.2 Text outputs

file	content
_lines.txt	line luminosities: SH95 H I and He II recombination lines (1640, 4686, ... when an He III zone exists), Porter He I lines (4471, 5876, 6678, 7065, 10830, ...) + the n -level collisional lines (+ hc collisional H I Balmer rows with h_coll_effects); columns elem, stage, $\lambda[\text{\AA}]$, L , $L/L(\text{H}\beta)$
_emis.h5	(with emis_output) leaf-by-leaf line emissivities: emis_h (SH95 lines) and emis_<el>_<stage> blocks [$\text{erg s}^{-1} \text{cm}^{-3}$; $\sum \text{emis} \cdot V = L$, rows match _lines.txt], wl_* wavelengths [$\text{\AA}$], plus LeafXYZ, LeafSize, n_H, n_e, T_e, x_HI/HeI/HeII, x_<el>_stages — self-contained for line maps
_nebcont.txt	nebular continuum L_ν (fb+ff tables, Milne, two-photon)
_dustir.txt	equilibrium-mixture dust IR spectrum (1–1000 μm) with the energy-closure header
_dustsed.txt	SEDust stochastic dust SED with PAH bands (when dust_sed)
_dustemis.h5	(with dust_emis_bands) SEDust band emissivities of each leaf, same block layout as _emis — readable by EmisData for dust-band maps
_image.h5	(with ion_peel) peel-off images for each observer: direct_ion/scatt_ion (+_fuv) [$\text{erg s}^{-1} \text{cm}^{-2} \text{sr}^{-1}$], suffix _obs<k> when several observers; with peel_bins, direct_cube/scatt_cube (n_x, n_y, n_ν) and the E_bin/dE_bin/wl_bin bin-grid arrays; with save_direct0, the unattenuated direc0_ion/_fuv (or direc0_cube)

3.3 Reading outputs in Python

tools/python/mochii_output.py is an importable module (works directly in a notebook) that reads every section file (HDF5 or FITS; read_sections), the line table (read_lines_table), and the text spectra (read_spectrum), and builds 2D maps from the emissivity file:

```

import sys; sys.path.insert(0, ".../MoCHII/tools/python")
import matplotlib.pyplot as plt
from mochii_output import EmisData

d = EmisData("run_emis.h5")
d.info()                # blocks, fields, line count
d.line_list()           # every stored line (block, wl)

# ready-made figures (notebook): returns (fig, ax)
fig, ax = d.plot_line("o_3", 5007.)          # log S map
fig, ax = d.plot_line("h", 6563., unit="intensity",
                    photons=True)           # photons/s/cm^2/sr
fig, ax = d.plot_field("T_e", weight="EM")
fig, axs = plt.subplots(1, 2); d.plot_line("s_2", 6717., ax=axs[0]) \
    ; d.plot_line("s_2", 6731., ax=axs[1])    # doublet side by side

# raw arrays, when the figure is yours to build
img, ext = d.line_map("o_3", 5007., axis="z", npix=512)
img, ext = d.line_map("h", 6563., unit="intensity", photons=True)
img, ext = d.field_map("T_e", weight="EM")    # LOS-weighted average
Q = d.line_luminosity("h", 4861., photons=True) # photon rate [1/s]

```

Line maps are flux-conserving: each leaf's luminosity is deposited with exact area overlap onto the pixel grid (AMR leaf sizes handled), so $\sum S A_{\text{pix}} = L_{\text{line}}$ to machine precision. `unit='flux'` (default) gives surface brightness [$\text{erg s}^{-1} \text{cm}^{-2}$]; `unit='intensity'` gives specific intensity [$\text{erg s}^{-1} \text{cm}^{-2} \text{sr}^{-1}$] = $S/4\pi$ (isotropic emission, optically thin); `photons=True` converts either to photon units (divides by hc/λ of the line). A `slab=(lo,hi)` cut restricts the line of sight. Field maps average `T_e`, `n_e`, `x_HI`, ... along the line of sight with weight '`EM`' ($n_e^2 V$), '`ne`', '`nH`', or '`V`'. The same module doubles as a command-line tool for quick looks:

```

python3 mochii_output.py run_emis.h5          # list lines
python3 mochii_output.py run_emis.h5 --line o_3 5007 \
    --npix 512 --log --out o3.png
python3 mochii_output.py run_emis.h5 --line h 6563 \
    --unit intensity --photons                # photons/s/cm^2/sr
python3 mochii_output.py run_emis.h5 --field T_e --weight EM

```

3.4 Slices and cutouts

A *slice* is the actual leaf value on a plane (the cell that contains each pixel), distinct from the line-of-sight *projection* above — a slice resolves the ionization front sharply where a projection blurs it. `tools/python/leaf_field.py` holds

the backend (leaf_slice, the nearest-leaf fallback leaf_slice_nn, and plot_slice); mochii_output.py exposes it on the output files:

```
from mochii_output import EmisData, RatesData

r = RatesData("run_rates.h5")          # leaf state
fig, ax = r.plot_field_slice("T_e", axis="z", coord=0.0)
fig, ax = r.plot_field_slice("x_HI", axis="z", coord=0.0, log=True)
img, ext = r.slice_field("n_e", axis="x", coord=0.0,
                        bounds=(-2,2,-2,2)) # cutout window

d = EmisData("run_emis.h5")
fig, ax = d.plot_line_slice("o_3", 5007., axis="z") # emissivity slice
```

Exact slices need the leaf full widths: the _emis.h5 file always carries LeafSize, and _rates.h5 does too; a rates file written before LeafSize existed falls back to a nearest-leaf slice (a one-time note is printed). The command line takes --slice FIELD --axis z --coord 0 on either file.

3.5 Building AMR grids

tools/python/AMR_grid/ builds the generic AMR octree file the code reads (grid_type='amr'): AMR_grid.py (the AMRGrid class — uniform, density-gradient, or resolution refinement; set_density/set_neutral_fraction; write to HDF5/FITS/.fits.gz/text with columns x,y,z, level,nH[,metallicity,xHI,ndust]) and make_amr_sphere.py (a sphere/shell CLI). AMRGrid carries the same slice/plot_slice/cutout so a grid can be inspected before a run.

4. Worked examples (the regression tests)

Each tests/ directory is self-contained: grid builder or a symlinked grid, the input file, and a check_*.py gate script.

directory	what it demonstrates / gate
g0_gamma	rate integrals vs. analytic attenuation (bias +0.004%)
g1_stromgren	Strömgren sphere, case B fixed T_e ; AMR $\equiv$ uniform
g2a_thermal	thermal H/He nebula vs. the 1D exact reference
g2_hii	MOCASSIN Lexington HII20/HII40 benchmarks + line fluxes
g4_refine	I-front re-refinement vs. native level 7
g4_tng	Illustris-TNG post-processing demo (shadows, maps)
d_dusty	dusty Strömgren; scattering bracket; T_d /IR
peel	peel-off images vs. the optically thin analytic flux (+0.017%); bin cubes
uni_dda	uniform grid: shared walk $\equiv$ octree (bit-identical); DDA walk to rounding
pdr	FUV/PDR switches: carbon-electron shell, photoelectric T_e

A minimal thermal + metals + dust input (from d_dusty):

¶meters

```

par%no_photons = 4.0e7          par%iseed = 1234
par%grid_type = 'amr'          par%amr_file = 'amr_L6.fits'
par%distance_unit = 'pc'       par%dust_model = 'global_dgr'
par%use_ion_band = .true.      par%nnu_ion = 32
par%eion_min = 13.598          par%eion_max = 100.0
par%tstar = 4.0e4              par%luminosity = 3.178e38
par%xHI_init = 1.0e-3          par%He_abund = 0.1
par%gas_niter = 60             par%gas_tol = 2.0e-3
par%solve_te = .true.          par%atomic_dir = '../data/atomic'
par%use_metals = .true.
par%ion_add_dust = .true.
par%ion_dust_kext = '../data/kext_albedo_WD_MW_3.1_60_D03.all_2009'
par%cext_dust = 4.868e-22      par%DGR = 1.0
par%out_file = 'run.h5'        par%file_format = 'hdf5'

```

/

5. Adding an ion

Adding an ion touches only data:

1. `tools/fitting/fit_cooling.py`: add the ion with a T_i ladder guessed from its `.elvlc` level energies; run; check the reported max error and the overlay figure.
2. `tools/fitting/fit_nlevel.py`: add the ion with the level count needed for its diagnostics; run.
3. `tools/fitting/make_element_data.py`: add the element to `ELEMENTS` (Z , stage count) and its thresholds; charge exchange entries where sources exist; run. The CHIANTI `.rrparams/.drparams` types 1–3 are handled (RR/RR2/DR/DR2 lines).
4. Fortran: a `par%abund_<El>` field and one entry in the `species_setup` name/abundance lists (only for a NEW element; new stages of a known element need nothing).
5. Verify: extend `verify_nlevel_pyneb.py` with a diagnostic ratio where PyNeb has data; run a smoke test and check the new lines in `_lines.txt`.

Data caveats met so far: Ar II has no CHIANTI v11 level/collision data (stage tracked without lines); Fe III collision strengths differ from PyNeb's BB14 by up to $\sim 16\%$ in 4658/4702 (known spread for iron); charge-exchange stage labels differ between the Huang (2023) and MOCASSIN transcriptions for Mg/Fe (the Huang labels are adopted; see `make_element_data.py`).

6. Practical notes

- **Bins:** 32 bins give $\sim 2\%$ rate-integral discretization; use 64 for sub-percent work. The `add_fuv` extension carries its own bins below `eion_min`, so the ionizing resolution is unaffected.
- **FUV and the PDR:** without `add_fuv` there is no radiation beyond the I-front and the metals there recombine toward neutral; with it, FUV penetrates the neutral gas (dust extinction only) and produces the PDR-side C II and Mg II. Metal photoheating and grain photoelectric heating are not in the thermal balance, so PDR-zone temperatures are indicative only.
- **Convergence:** use `conv_crit = 'vol'` for production — the cell-max criterion stalls at the front-cell noise floor (and, with `add_fuv`, at no-heating cells where T_e pins to `te_min`). Both measures are printed each iteration. The explicit diffuse field smooths the front and converges far better.
- **Starts:** prefer an ionized initial state; a neutral start advances the front $\sim$ one cell per iteration.
- **text tables:** must cover the band (the `astrodust` table stops at $912 \text{ \AA}$; use the `D03` table for EUV work). Tables are sorted on read (the `D03` file has an out-of-order seam).
- **Re-refinement:** one event per run; the old shared-memory windows are reclaimed only at exit.
- **SEDust:** `sed_workdir` must point at a `SEDust sed/` tree; the exact stochastic solve costs $\sim 0.05\text{--}0.1$ s for each gas leaf (distributed over ranks).